

Let It Try First: Uncertainty-Guided Retrieval Switching for Retrieval-Augmented Question Answering

Xiaohai Wang

School of Software, Yunnan University
Kunming, Yunnan, China
wangxiaohai@stu.ynu.edu.cn

Wei Zhou

School of Engineering, Yunnan University
Kunming, Yunnan, China
zwei@ynu.edu.cn

Yu Liang

School of Software, Yunnan University
Kunming, Yunnan, China
yuliang@ynu.edu.cn

Sixing Wu*

School of Software, Yunnan University
Kunming, Yunnan, China
wusixing@ynu.edu.cn

Abstract

Adaptive Retrieval-Augmented Generation (RAG) can dynamically select the best retrieval strategy for each query, thereby balancing answer quality and efficiency. However, existing approaches either follow fixed multi-stage pipelines or use a single-pass classifier to determine the strategy, leading to irreversible decisions once the strategy is selected. To address this limitation, we propose TF-RAG (Try First), an adaptive RAG framework for question-answering that first attempts to answer the question with the initial retrieval strategy, and can be dynamically switched to a stronger strategy if needed. To this end, TF-RAG monitors token-level entropy signals during generation, allowing the model to recognize when its confidence is low. This uncertainty-guided switching strikes a balance between accuracy and computational efficiency, while requiring only a lightweight routing classifier and no modifications to the underlying model architecture. Experiments across six different QA benchmarks show that TF-RAG consistently improves answer correctness while substantially reducing retrieval cost.

CCS Concepts

• Computing methodologies → Natural language generation.

Keywords

RAG; Question Answering; Adaptive Retrieval

ACM Reference Format:

Xiaohai Wang, Yu Liang, Wei Zhou, and Sixing Wu. 2026. Let It Try First: Uncertainty-Guided Retrieval Switching for Retrieval-Augmented Question Answering. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3774904.3792870>

1 Introduction

Large language models (LLMs) have demonstrated remarkable performance in question-answering (QA) [3]. Nevertheless, LLMs are

prone to generating occasional hallucinatory answers that sound plausible yet are factually inaccurate, which may risk misleading users and propagate erroneous knowledge. To this end, researchers have proposed Retrieval-Augmented Generation (RAG) [6] to integrate retrieved external knowledge to augment LLMs' knowledge reserves and mitigate the hallucination. Although recent RAG works have achieved notable results, they still struggle to balance performance and efficiency because they blindly execute the computationally expensive knowledge retrieval process for all given questions, overlooking the potential of LLMs' built-in parametric knowledge, an inherently more efficient way for simple questions.

Intuitively, the unnecessary introduction of external knowledge will inevitably damage the overall computational efficiency. Addressing this challenge, adaptively determining whether and how to incorporate RAG into the QA process remains non-trivial. Recently, DRAGIN [9] leverages model uncertainty regarding a given question to decide when retrieval is necessary. However, it relies exclusively on empirically defined thresholds and fails to account for the trade-off between answer quality and efficiency. To jointly optimize these two key objectives, Adaptive-RAG [5] employs a classifier to assess query difficulty, then selects the optimal generation strategy from direct answering, Naïve RAG, and IRCot [10]. Nevertheless, Adaptive-RAG makes a one-time strategy decision, failing to dynamically adjust strategies after the prediction.

Consequently, we propose a novel **TF-RAG** (*Try First RAG*) to flexibly switch from using the efficient yet weaker parametric knowledge to the more powerful yet costly external knowledge retrieval at the right time. The core idea is to first predict an initial answering strategy suited to the given question, and escalate to retrieval-enhanced strategies only when the initial strategy proves insufficient. Specifically, we introduce a *Strategy Router (Trier)* that leverages prior questions and their corresponding optimal strategies to select an initial answering strategy for the given question. During the answer generation, *TF-RAG* incorporates a *Strategy Upgrader (Thinker)* that monitors the LLM's internal state in real time; whenever the estimated uncertainty exceeds a predefined computational efficiency threshold, it triggers an upgrade to a higher-cost but more powerful strategy, thereby ensuring the answer quality.

Extensive experiments show that *TF-RAG* achieves superior performance on six QA datasets while maintaining reasonable computational cost. Further analysis confirms that *TF-RAG* provides a scalable and adaptive solution for RAG strategy selection.

*Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates.*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2307-0/2026/04

<https://doi.org/10.1145/3774904.3792870>

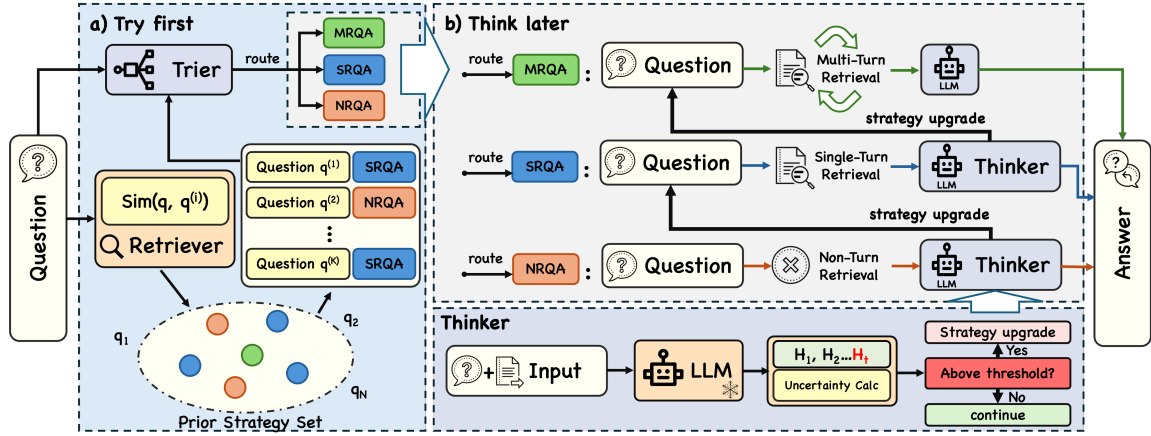


Figure 1: The overview of TF-RAG. Trier selects an initial QA strategy for a given question, while Thinker monitors the LLM’s uncertainty during the execution of the strategy to determine whether a strategy upgrade is necessary.

2 Method

2.1 Preliminary

As introduced in Adaptive-RAG [5], questions of varying complexity require different strategies to balance performance and efficiency. Thus, we define a 3-scale strategy set $S = \{s_{nr}, s_{sr}, s_{mr}\}$:

1. *Non-Retrieval QA (NRQA, s_{nr})*: It answers the question using the LLM (f_θ) and the corresponding prompt¹ instruction without any retrieval. Formally, for a question q , the strategy produces the answer $\hat{a}$ only based on the parametric knowledge:

$$\hat{a} = f_\theta(q). \quad (1)$$

2. *Single-Turn Retrieval QA (SRQA, s_{sr})*: It performs one round of retrieval. Additionally given an knowledge base C , LLMs generate the answer with the documents D retrieved by the retriever R :

$$\hat{a} = f_\theta(q, R(q, C)). \quad (2)$$

3. *Multi-Turn Retrieval QA (MRQA, s_{mr})*: It performs multi-round retrieval. Following the CoT paradigm of IRCot [10], at each step $t \in [1, T]$, the model first generates a reasoning c_t , and then this reasoning is used to retrieve the next document set d_t :

$$d_t = R(c_t, C), \quad c_t = f_\theta(q, d_{t-1}, c_{t-1}), \quad (3)$$

where this iterative process continues until a stopping condition is met (e.g., a maximum number of steps is reached or the reasoning chain contains an explicit answer cue). Finally, f_θ generates the answer with the full reasoning chain and all retrieved documents:

$$\hat{a} = f_\theta(q, \{d_1, d_2, \dots, d_T\}, \{c_1, c_2, \dots, c_T\}). \quad (4)$$

2.2 Overview

Intuitively, the three strategies exhibit a progressive trade-off. Generally, answer accuracy increases from s_{nr} to s_{mr} , but so does the computational cost. As shown in Figure 1, we propose *TF-RAG*, which first uses *Strategy Router (Trier)* to select the initial strategy based on question complexity and then *Strategy Upgrader (Thinker)*

makes the on-demand (optional) secondary routing decisions based on the LLM’s dynamic internal state during generation.

2.3 Strategy Router (Trier)

To predict the initial answering strategy $\hat{s} \in S$ for a given question q , our first-stage predictor *Trier* is implemented as a *Prior Strategy Retrieval Enhanced* classifier, using the existing queries and their strategy labels to help the current prediction.

In detail, *Trier* first retrieves the top- k existing questions $Q^{PSR} = \{q^{(1)}, q^{(2)}, \dots, q^{(k)}\}$ that are most similar to the current question q from the training set Q_{train} as the prior strategy pairs, where similarity is computed as cosine similarity between sentence embeddings. The optimal strategy labels $s^*(q^{(i)})$ carried by questions $q^{(*)} \in Q^{PSR}$ can provide additional signals to assist the current probability distribution prediction $\hat{p}(q)$ over all strategy classes:

$$\hat{p}(q) = \text{softmax}(\mathbf{W}h_{CLS} + \mathbf{b}), \quad (5)$$

$$h_{CLS} = \text{BERT}\left(q, \{q^{(i)}, s^*(q^{(i)})\}^k\right), \quad (6)$$

where the representation h_{CLS} is obtained from the BERT encoder. Then, the distribution $\hat{p}(q)$ is computed via a linear layer and a softmax function. The final predicted strategy $\hat{s}(q)$ is selected by:

$$\hat{s}(q) = \arg \max_{s \in S} \hat{p}(q). \quad (7)$$

Training strategy. For each question $q_i \in Q_{train}$, let p_i be the probability of the ground-truth strategy $s^*(q_i)$. Then, we adopt Focal Loss [7] to mitigate label imbalance during the training:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \alpha_i (1 - p_i)^\gamma \log(p_i), \quad (8)$$

where α_i is the class-balancing weight according to $s^*(q_i)$ to adjust label imbalance and γ controls the focusing strength, reducing the relative loss contribution of well-classified samples.

2.4 Strategy Upgrader (Thinker)

If the first-stage predictor *Trier* has selected a low-cost strategy $\hat{s} \in \{s_{nr}, s_{sr}\}$, *Thinker* will track whether the this strategy is suitable

¹Prompt Template: <https://anonymous.4open.science/r/TF-RAG-C45E>

enough or needs to be upgraded based on the uncertainty exhibited by LLMs during generation, i.e, the generation entropy H_t :

$$H_t = - \sum_{v \in V} p_\theta(v | \mathcal{I}, \tilde{y}_{<t}) \log p_\theta(v | \mathcal{I}, \tilde{y}_{<t}), \quad (9)$$

where p_θ denotes the LLM’s probability of token v over the full vocabulary V at time t ; $\mathcal{I}$ is the current input context consisting of the prompt template, the question q , and the optionally retrieved documents D ; $\tilde{y}_{<t}$ is previously generated answer tokens.

Then, if the current entropy H_t exceeds the strategy-aware threshold $\tau(\hat{s})$, the current strategy $\hat{s}$ is deemed insufficient and will be upgraded immediately to regenerate the answer:

$$\pi_{\text{Thinker}}(q, \hat{s}(q)) = \begin{cases} \hat{s}, & \forall t, H_t \leq \tau(\hat{s}), \\ \text{Upgrade}(\hat{s}), & \exists t, H_t > \tau(\hat{s}). \end{cases} \quad (10)$$

Once an upgrade is triggered, $\text{Upgrade}(s)$ selects the next strategy (one step forward) on the $s_{nr} \rightarrow s_{sr} \rightarrow s_{mr}$ path, and LLM will not revert to the older strategies. After switching strategies, the LLM will restart answer generation under the new strategy.

Entropy Threshold. Rather than using a fixed entropy threshold $\tau(\hat{s})$ to control strategy upgrades, we evaluate the thresholds $\tau(s_{nr})$ for $s_{nr} \rightarrow s_{sr}$ and $\tau(s_{sr})$ for $s_{sr} \rightarrow s_{mr}$ by assessing their respective impacts on computational efficiency. For each question $q_i \in Q_{\text{train}}$, we compute the maximum token entropy $H_{i,\hat{s}}$ of each strategy $\hat{s} \in \{s_{nr}, s_{sr}\}$. For a given entropy value $H_{i,\hat{s}}$, we design the upgrade rule by assuming that all questions whose $H_{j,\hat{s}}$ is higher than $H_{i,\hat{s}}$ can trigger upgrade to the next strategy. By evaluating all candidate $H_{i,\hat{s}}$ on training set, we select the one that maximizes the objective as the optimal threshold $\tau(\hat{s})$:

$$\tau(\hat{s}) = \arg \max_{H_{i,\hat{s}}} E(H_{i,\hat{s}}), \quad (11)$$

$$E(H_{i,\hat{s}}) = \frac{\Delta M(H_{i,\hat{s}})}{\Delta RC(H_{i,\hat{s}}) + \beta} \cdot \log(U(H_{i,\hat{s}}) + 1), \quad (12)$$

where $E(H_{i,\hat{s}})$ is the efficiency score we defined, ΔM represents performance gain measured as the sum of the evaluation metrics (EM, F1, and ACC), ΔRC denotes the increase in retrieval cost, β is smoothing parameter and U is the number of upgraded questions surpass $H_{i,\hat{s}}$. β and U are introduced to prevent the efficiency score $\frac{\Delta M(H_{i,\hat{s}})}{\Delta RC(H_{i,\hat{s}})}$ from being distorted by boundary effects. If $\tau(\hat{s}) < \epsilon$, where ϵ is the validity threshold, the performance gain from upgrading is considered insufficient because the improvement per additional retrieval is too small, and the path is discarded.

2.5 Strategy Labeling

To determine the optimal strategy in the training stage, we propose *Semantic Similarity Labeling (SSL)*. Given a question $q_i \in Q_{\text{train}}$, the answer set generated by the strategy set S is denoted as $A(q_i) = \{\hat{a}_{nr}^{(i)}, \hat{a}_{sr}^{(i)}, \hat{a}_{mr}^{(i)}\}$. We evaluate the correctness of each strategy based on the semantic similarity $\text{Sim}(\hat{a}_s^{(i)}, a_i)$, using the embedding model between the generated answer $\hat{a}_s^{(i)}$ and the ground-truth answer a_i . The optimal strategy label for q_i is determined by selecting the strategy that achieves the highest semantic similarity score. In cases where multiple strategies yield the same highest similarity, the one with the lowest computational cost is chosen.

3 Experiments

3.1 Setting

Datasets. We use three single-hop datasets (SQuAD, Natural Questions, TriviaQA) for simpler QA tasks and three multi-hop datasets (MuSiQue, HotpotQA, 2WikiMultiHopQA) for more challenging scenarios. For each dataset, we sampled 1,500 instances for the training set and 500 instances for the validation and test sets. And we set efficiency validity threshold $\epsilon = 0.5$ and smoothing parameter $\beta = 5$ for all datasets.

Metrics. Four metrics were employed: (1) **Exact Match (EM)**, which checks whether LLMs’ prediction exactly matches the ground truth; (2) **F1** score, which quantifies the overlap between LLMs’ prediction and the ground truth; and (3) **Accuracy (ACC)**, which evaluates whether LLMs’ prediction contains the ground truth; (4) **Retrieval Count (RC)**, which reflects, to a certain extent, the increase in contextual overhead, predicted tokens, and LLM calls.

Baselines and Models. We compare our method against the following baselines. The strategy set S are evaluated: **NRQA**, **SRQA** and **MRQA** [10]. The **Adaptive RAG** methods considered include: (1) **Adaptive Retrieval** [8] and **DRAGIN** [9], which trigger retrieval based on preset thresholds; (2) **Self-RAG** [1], in which LLaMA2-7B is trained to decide whether retrieval is needed; (3) **Adaptive-RAG** [5], which uses an external classifier trained with SSL for comparison; and (4) **SKR_{knn}** [11], which triggers retrieval by leveraging a KNN approach. We employ **Flan-T5-XL** [2] and **LLaMA3-8B-Instruct** [4] as backbones, **Qwen3-Embedding-0.6B** for embedding, and **BERT-base-cased** for *Trier*.

Table 1: Averaged results on six datasets. Bold indicates the best in routing methods (three singleton **QA are excluded).

Method	Flan-T5-XL				LLaMA3-8B-Instruct			
	EM	F1	Acc	RC	EM	F1	Acc	RC
NRQA	0.147	0.205	0.157	0.00	0.272	0.350	0.293	0.00
SRQA	0.342	0.436	0.382	1.00	0.321	0.415	0.378	1.00
Adaptive Retrieval	0.248	0.327	0.275	0.50	0.306	0.395	0.347	0.50
Self-RAG*	0.103	0.218	0.286	0.48	0.103	0.218	0.286	0.48
DRAGIN	0.200	0.275	0.237	2.98	0.292	0.399	0.387	2.13
SKR _{knn}	0.301	0.386	0.338	1.78	0.335	0.428	0.384	1.31
Adaptive-RAG	0.318	0.413	0.357	0.94	0.331	0.423	0.372	0.70
TF-RAG (Ours)	0.356	0.452	0.401	1.58	0.364	0.462	0.414	1.40
MRQA	0.384	0.482	0.431	4.91	0.362	0.461	0.425	3.59

3.2 Results

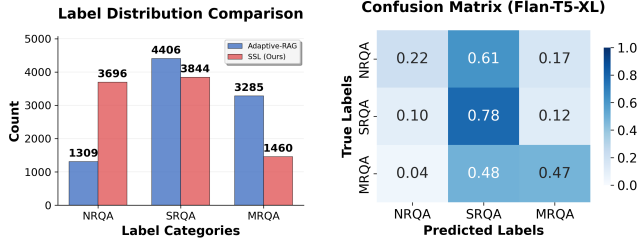
3.2.1 Main Results. We present in Table 1 the averaged test results of Flan-T5-XL and LLaMA3-8B-Instruct across six QA datasets. NRQA, SRQA, and MRQA exhibit the performance and cost of each singleton strategy. Compared with the uncertainty-hyperparameter-based method DRAGIN, our approach achieves better QA performance while requiring fewer retrievals. In addition, relative to the multi-classification-based SKR_{knn} and Adaptive-RAG, it delivers superior QA accuracy while maintaining the computational cost within a reasonable range. On LLaMA3-8B-Instruct, our method reaches an EM of 0.364, outperforming MRQA with a lower retrieval

Table 2: Ablation study on Flan-T5-XL

Method	EM	F1	Acc	RC
TF-RAG (Ours)	0.356	0.452	0.401	1.58
w/o Trier	0.295	0.383	0.326	1.16
w/o Thinker	0.324	0.419	0.370	1.02
w/o Thinker + PSR	0.292	0.380	0.326	0.74

cost (RC 1.40 vs. 3.59). It demonstrates that a strategy-routing approach can outperform a single, highly complex QA strategy.

3.2.2 Ablation Study. To investigate the contribution of different components in our method, three experimental modes were designed. As shown in Table 2, removing *Trier* leads to a substantial performance drop, removing *Thinker* (w/o Thinker) further reduces EM (0.324 vs. 0.356), and removing both *Thinker* and *PSR* yields the worst performance (EM 0.292, RC 0.74). These results indicate that these modules play a complementary role in enhancing answer accuracy while maintaining efficiency.

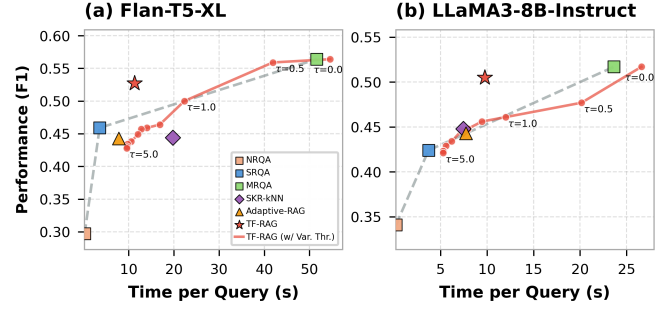
**Figure 2: Label distributions for SSL vs. Adaptive-RAG (left) and confusion matrix (right) on Flan-T5-XL**

3.2.3 Distribution Analysis. We compare SSL with the labeling approach used in Adaptive-RAG, as shown on the left of Figure 2. It can be observed that our SSL maintains QA quality while placing greater emphasis on reducing computational overhead. The right side of Figure 2 illustrates the strategy classification results of *TF-RAG*. *TF-RAG* performs reasonably well in selecting SRQA, but there remains room for improvement in distinguishing between SRQA and NRQA as well as between MRQA and SRQA. This potential improvement will be explored as a direction for future research.

3.2.4 Efficiency Analysis. The relationship between QA performance and average time is shown in Figure 3. Our method achieves higher efficiency than *SKR_{KNN}* and Adaptive-RAG. In addition, we conduct an analysis in which the upgrade thresholds $\tau(\$)$ are not computed but instead treated as the hyperparameter which noted as *TF-RAG* (w/ Var. Thr.). We set both thresholds to values from 0.0 to 5.0 and report the performance. This comparison with our method highlights the effectiveness of our computed threshold.

4 Conclusion

We propose the *TF-RAG*, which first selects an initial answering strategy to generate an answer and then monitors the LLM’s internal states during the answering process to determine whether an

**Figure 3: The QA performance (F1) and efficiency (Time/Query) of different methods on 2WikiMultiHopQA.**

upgrade to a different strategy is needed. Experiments on multiple datasets demonstrate that *TF-RAG* outperforms baseline methods while maintaining reasonable computational cost. Future work will focus on refining the classification mechanism to further improve decision accuracy and reduce cost.

Acknowledgments

This work has been supported by the Yunnan Research Project (Grant Nos. 202503AG380006, 202401AT070474, 202501AU070059, 202403AP140021), National Natural Science Foundation of China (Grant Nos. 62562061, 62502422 and 62462067), and Yunnan Provincial Department of Education Science Research Project (Grant Nos. 2025J0006, 2024J0010 and 2025J0007).

References

- [1] Akari Asai, Zeqiu Wu, Yizhong Wang, et al. 2024. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. In *The Twelfth International Conference on Learning Representations, ICLR 2024*.
- [2] Hyung Won Chung, Le Hou, Shayne Longpre, et al. 2024. Scaling instruction-finetuned language models. *Journal of Machine Learning Research* 25, 70 (2024), 1–53.
- [3] Yunfan Gao, Yun Xiong, Xinyu Gao, et al. 2023. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997* 2, 1 (2023).
- [4] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783* (2024).
- [5] Soyeong Jeong, Jinheon Baek, Sukmin Cho, et al. 2024. Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity. In *2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL 2024*. 7029–7043.
- [6] Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6–12, 2020, virtual*.
- [7] Tsung-Yi Lin, Priya Goyal, Ross Girshick, et al. 2017. Focal loss for dense object detection. In *Proceedings of the IEEE international conference on computer vision*. 2980–2988.
- [8] Alex Mallen, Akari Asai, Victor Zhong, et al. 2023. When Not to Trust Language Models: Investigating Effectiveness of Parametric and Non-Parametric Memories. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 9802–9822.
- [9] Weihang Su, Yichen Tang, Qingyao Ai, et al. 2024. DRAGIN: Dynamic Retrieval Augmented Generation based on the Real-time Information Needs of Large Language Models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 12991–13013.
- [10] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, et al. 2023. Interleaving Retrieval with Chain-of-Thought Reasoning for Knowledge-Intensive Multi-Step Questions. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 10014–10037.
- [11] Yile Wang, Peng Li, Maosong Sun, et al. 2023. Self-Knowledge Guided Retrieval Augmentation for Large Language Models. In *Findings of the Association for Computational Linguistics: EMNLP 2023*. 10303–10315.